

Data Warehouse Integration



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://about.softuni.bg>

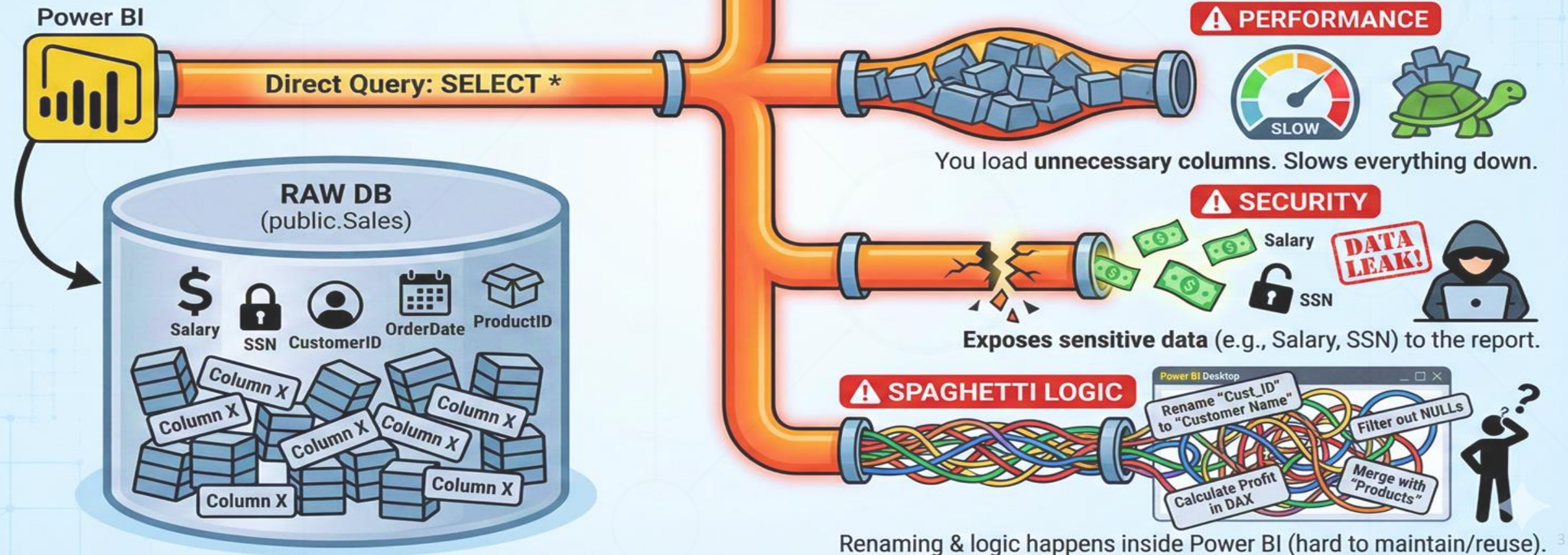
1. **The "Spaghetti" Problem:** Why connecting directly is bad.
2. **The Solution:** SQL Views as an Abstraction Layer.
3. **PostgreSQL specifics:** Syntax & Best Practices.
4. **The Engine:** Power Query & M.
5. **Performance:** Understanding Query Folding.
6. **Engineering:** Parameters & Dynamic Connections.



THE ROOKIE MISTAKE: Connecting to Raw Tables

The Rookie Mistake

Scenario: You connect Power BI directly to the Sales table. "SELECT * FROM public.Sales"



The Rookie Mistake

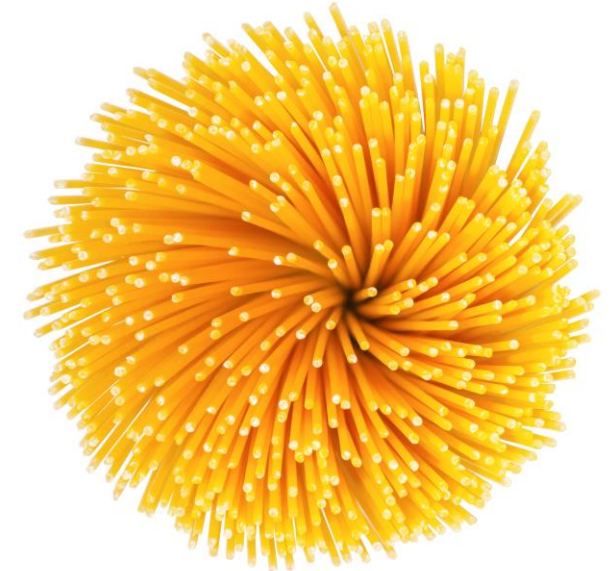
- **Scenario:** You connect Power BI directly to the raw table `public.sales_data`.
- **The Query:** `SELECT * FROM sales_data`.
- **Result:** You import 50 columns, even if you need only 3.



Why is SELECT * Dangerous?

- **Performance:** Wastes memory and network bandwidth.
- **Security:** Might expose salary, personal_id, or profit_margin to unauthorized users.
- **Fragility:** If the database admin renames a column, your report crashes.

- **Scenario:** You rename "cust_nm" to "Client Name" inside Power BI.
- **Problem:** You have to do this in every single report.
- **Result:** Inconsistent logic (Spaghetti code).



The Solution: Upstream Thinking

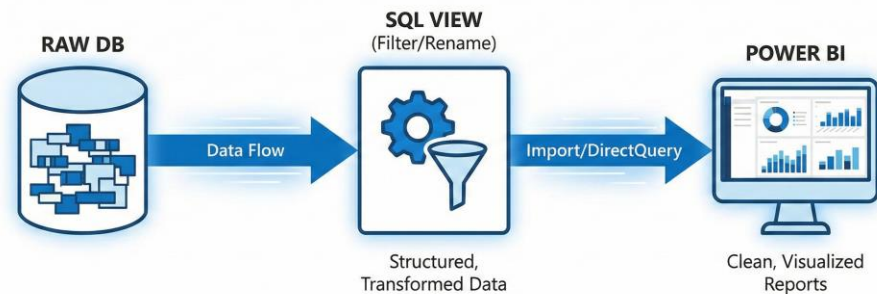
- **Downstream (Bad):** Fixing data in Power BI.
- **Upstream (Good):** Fixing data in the Database (SQL).
- **Rule:** Push the logic as close to the source as possible.



The Abstraction Layer

Definition: A shield between the messy database and the clean report.

Implementation: We use SQL Views.



Live Demo



What is a View?

- **Definition:** A "Virtual Table".
- It takes 0 bytes of storage space.
- It is simply a saved query that runs every time you request it.



Benefits of Views

1. **Security:** Expose only specific columns.
2. **Simplicity:** Hide complex JOINS from the end user.
3. **Consistency:** Calculate **Total Revenue** once in SQL, use it everywhere.



Syntax: Creating a View

Standard SQL:

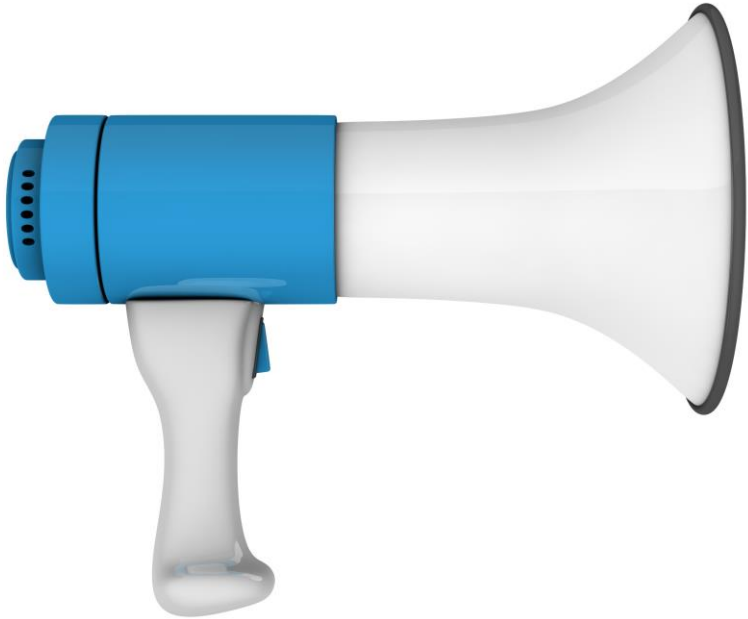
SQL

```
CREATE VIEW view_name AS  
SELECT column1, column2  
FROM table_name;
```

Best Practice:

Always use **OR REPLACE**
to easily update it later.





Warning: PostgreSQL converts everything to lowercase unless you use quotes.

Select Name From Users -> looks for name in users.

Select "Name" From "Users" -> looks for Name in Users.

Handling Quotes in Views

- If your columns have spaces or CamelCase,
- you **MUST** use double quotes:

SQL

```
SELECT
    first_name AS "First Name", -- Clean name for Power BI
    last_name AS "Last Name"
FROM employees
```

-- ❌ Without quotes (PostgreSQL converts to lowercase)

```
SELECT Name FROM Users → searches for: name, users
```

-- ✅ With quotes (preserves case)

```
SELECT "Name" FROM "Users" → searches for: Name, Users
```

- Use prefixes to organize your views.
- vw_dim_ -> Dimension (Customers, Products, Locations).
- vw_fct_ -> Fact (Sales, Orders, Transactions).
- **Example:** vw_dim_Customers, vw_fct_Orders.

Naming Conventions

Your view names should tell a story.

- **vw_dim_Customer**
 - *vw* = View
 - *dim* = Dimension (Descriptive data: Who, Where)
- **vw_fct_Sales**
 - *fct* = Fact (Transactional data: Numbers, Dates)
- **Golden Rule:** Column names in the View should be user-friendly.
- Avoid: `t_stamp_cr`
- Use: `Creation Date`



Think Like
an
Engineer

Transformation in SQL

- Don't clean in Power BI if you can clean in SQL.
- Don't just select. Transform!
- Concatenation: `first_name || ' ' || last_name`
- Math: `qty * price`
- Filtering: `WHERE is_active = true`



Bad Practice

(Doing it in Power Query):

- Renaming `cust_fname` -> First Name
- Calculating `Price * Qty`

✗ Power Query M code (messy)

✓ SQL View (clean)

Good Practice

(Doing it in the View)

SQL

```
CREATE VIEW vw_Order_Details AS
SELECT
    o.order_id,
    c.customer_name AS "Client", -- Renaming
    o.qty * o.price AS "Total Revenue" -- Calculation
FROM orders o
JOIN customers c ON o.cust_id = c.id
```



Software
University

Live Demo



What is Power Query?

- It is the **Kitchen** of Power BI.
 - It extracts, cleans, and prepares data before it goes to the dashboard.
 - *Note: It is the same engine used in Excel.*
-
- **Tool:** Power BI Desktop.
 - **Engine:** Power Query.

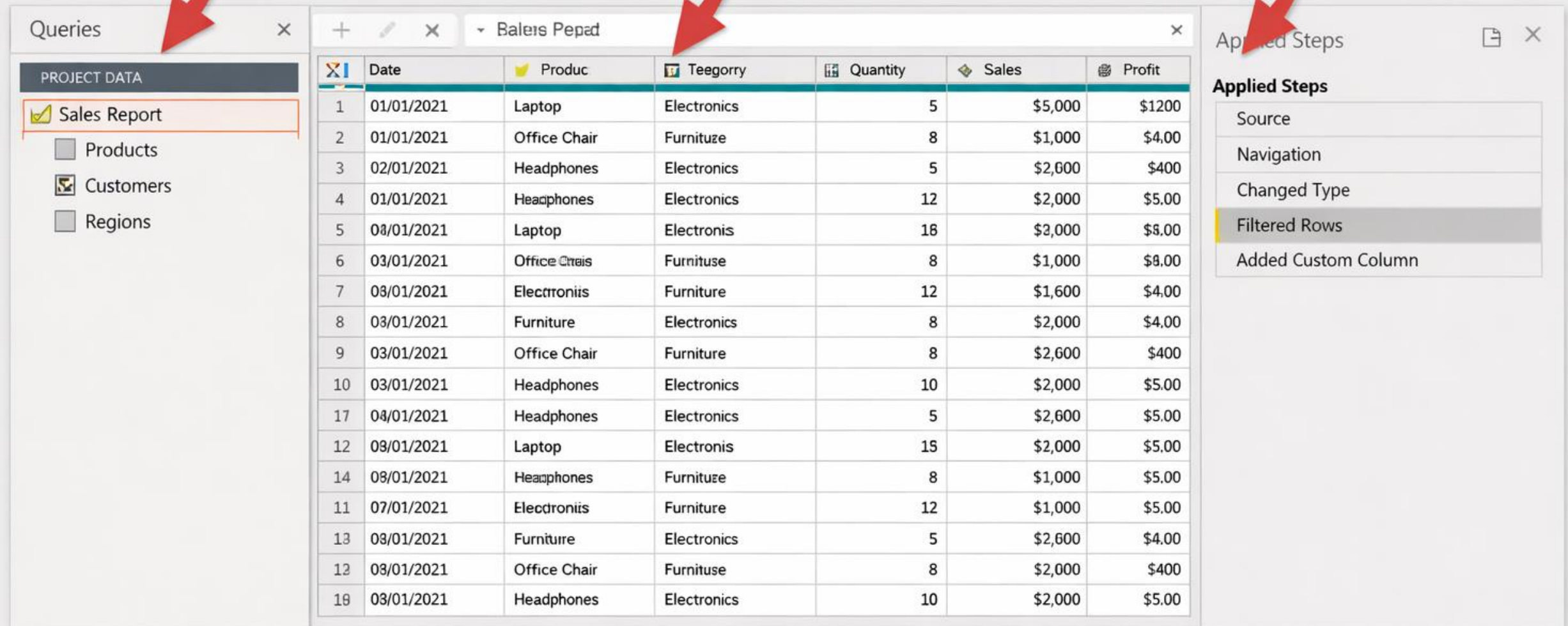


The Interface

1. Queries Pane

2. Data Preview

3. Applied Steps



The interface displays three main panels:

- Queries Pane:** Shows a list of queries under the heading "PROJECT DATA". The "Sales Report" query is selected and highlighted. Below it, there are checkboxes for "Products", "Customers", and "Regions".
- Data Preview:** Displays a table of data for the "Sales Report" query. The table has columns: Date, Product, Category, Quantity, Sales, and Profit. The data is filtered to show only rows where the "Sales" value is greater than \$1,000.
- Applied Steps:** Shows a list of steps applied to the query. The steps are: Source, Navigation, Changed Type, Filtered Rows, and Added Custom Column. The "Filtered Rows" step is currently selected.

	Date	Product	Category	Quantity	Sales	Profit
1	01/01/2021	Laptop	Electronics	5	\$5,000	\$1200
2	01/01/2021	Office Chair	Furniture	8	\$1,000	\$400
3	02/01/2021	Headphones	Electronics	5	\$2,600	\$400
4	01/01/2021	Headphones	Electronics	12	\$2,000	\$5.00
5	03/01/2021	Laptop	Electronics	18	\$3,000	\$8.00
6	03/01/2021	Office Chair	Furniture	8	\$1,000	\$8.00
7	03/01/2021	Electronics	Furniture	12	\$1,600	\$4.00
8	03/01/2021	Furniture	Electronics	8	\$2,000	\$4.00
9	03/01/2021	Office Chair	Furniture	8	\$2,600	\$400
10	03/01/2021	Headphones	Electronics	10	\$2,000	\$5.00
11	04/01/2021	Headphones	Electronics	5	\$2,600	\$5.00
12	03/01/2021	Laptop	Electronics	15	\$2,000	\$5.00
13	08/01/2021	Headphones	Furniture	8	\$1,000	\$5.00
14	07/01/2021	Electronics	Furniture	12	\$1,000	\$5.00
15	03/01/2021	Furniture	Electronics	5	\$2,600	\$4.00
16	03/01/2021	Office Chair	Furniture	8	\$2,000	\$400
17	03/01/2021	Headphones	Electronics	10	\$2,000	\$5.00

The "M" Language

The "M" Language is the powerful code used to transform and prepare your data in Power Query.

```
let
    Source = Excel.CurrentWorkbook(){[Name="Sales"]}[Content],
    FilteredRows = Table.SelectRows(Source, each [Date] <= #date(2024, 4, 1)),
    GroupedData = Table.Group(FilteredRows, {"Product"}, {"Total Sales",
        SortedData = Table.Sort(GroupedData, {"Total Sales", Order.Descending})
    in SortedData
```



	Product	Total Sales
1	Computers	\$3,250
2	Phones	\$2,100
3	Tablets	\$1,850
4	Accessories	\$1,300

 **Powerful**

 **Flexible**

Customizable queries and logic

 **Functional**

Based on functions and expressions

- Every button you click generates code.
- This code is called M (Mashup Language).
- You don't need to write it fluently, but you need to read it.

Applied Steps = Time Travel

- You can click on any previous step to see how data looked then.
- You can insert, delete, or reorder steps.
- Warning: Deleting an early step might break later steps.



Connecting to PostgreSQL

- Get Data -> Database -> PostgreSQL.
- Server: localhost (or IP).
- Database: TechCorp_DB.
- Data Connectivity: Import (usually).

The Golden Rule

- "Filter Early, Filter Often."
- Remove data you don't need as soon as possible.



Live Demo



What is Query Folding?

Query Folding: The Performance Magic

Definition:

- The ability of Power Query to generate a single SQL statement that executes on the server side.
- Power Query translates your UI clicks into a native SQL Statement.
- It sends this SQL to the database.
- The Database does the work, not your laptop.

Translation:

- **You do:** Filter rows in Power BI (Keep last 2 years).
- **Power BI does:** Sends a `WHERE Year > 2023` to the database.
- **Result:** Only the relevant data travels over the network.

Analogy: The Pizza Order

Folded (Good): 👍

- Simple filters, renaming, basic math.
- Power BI writes SQL for you.



"I want a Pizza without onions."

✅ **Folded (Server-side):**
🍕 Chef removes onions → You get clean pizza
💾 DB filters **data** → Power BI gets clean **data**



Not Folded (Bad): 👎

- Writing complex custom SQL in the "Get Data" window.
- Using specific M functions (e.g., `Index Column`, some text parsing).
- Power BI downloads EVERYTHING and filters locally.



"I want a standard Pizza ."

❌ **Not Folded (Client-side):**
🍕 You manually remove onions → Wasted **time**
💾 Power BI filters locally → Wasted bandwidth



How to verify Folding?

Validating Query Folding

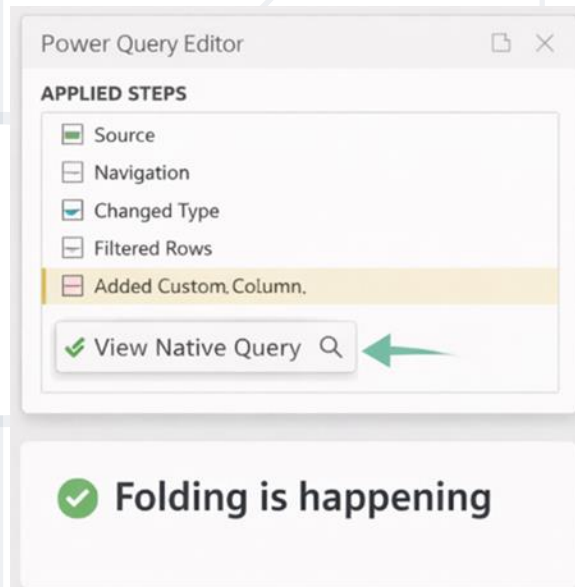
Steps:

1. Go to **Power Query Editor**.

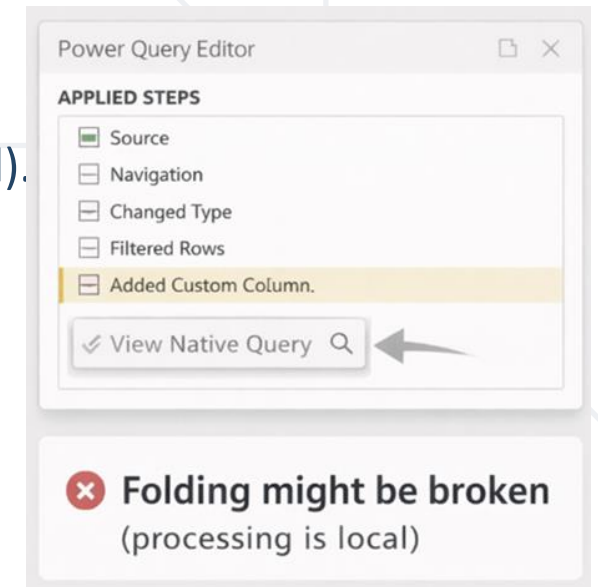
2. Right-click the last step in "Applied Steps".

3. Look for **"View Native Query"**.

If Clickable:
Folding is happening.



If Greyed Out:
Folding might be broken (processing is local).



What breaks Folding?

- Writing Custom SQL in the source settings.
- Using transforms that SQL doesn't support (e.g., complex text parsing, index columns).
- Mixing data sources (Excel + SQL).

The "Custom SQL" Trap

If you paste a SQL query in the "Get Data" window.....

Power BI treats it as a "Black Box".

It stops folding immediately.

Solution: Create a View in the DB instead!

✗ Don't do this:

[Get Data → PostgreSQL → "Advanced Options"]
`SELECT * FROM sales WHERE year = 2024`

Result: No query folding! ✗

✓ Do this instead:

1. `CREATE VIEW vw_sales_2024 AS SELECT * FROM sales WHERE year = 2024`
2. Power BI → Connect to `vw_sales_2024`

Result: Query folding works! ✓

Live Demo





Performance Checklist:

✓ Heavy logic in SQL Views

✓ Simple filters in Power Query (they fold)

✓ Check "View Native Query" regularly

✗ Avoid Custom SQL in source

✗ Avoid Index Column transforms

✗ Don't mix data sources (Excel + SQL)

Why care?

- **Folding:** 5 seconds refresh.
- **No Folding:** 5 minutes refresh + Laptop fan spinning + Memory crash.

1. Do heavy logic in SQL Views.
2. Do simple filters/renaming in Power Query (it folds).
3. Check "Native Query" often.

The Environment Problem

- You build a report connected to DEV_DB (Test data).
- Now you need to publish it for the CEO using PROD_DB (Real data).
- Do you manually edit 50 queries?



- **Definition:**
Variables that store configuration values.
- **Use them for:**
Server Name,
Database Name,
File Paths.

Creating a Parameter

Home Tab -> Manage Parameters -> New.

Name: ServerParameter.

Type: Text.Current

Value: localhost.

Using Parameters

Go to the Source step of your query.

Replace "localhost" with Parameter:
ServerParameter.

Hardcoding is bad. 

- Avoid typing the Server Name in every single query.

Best Practice: Use Parameters 

1. Create a Parameter in Power BI: `ServerName`, `DatabaseName`.
2. Use these parameters in your Source step.
3. **Benefit:** Easily switch between `DEV` (Test DB) and `PROD` (Live DB).

1. Least Privilege Principle

- The DB User for Power BI should have **READ ONLY** access.
- Never use sa (System Admin) or root credentials!

2. Sharing PBIX files

- Credentials are **NOT** stored in the .pbix file.
- When you send the file to a colleague, they must enter their own credentials to refresh.



Live Demo



- **Views:** Our primary way to ingest data.
- **Folding:** The key to performance. Let the DB work.
- **Parameters:** The key to flexibility.
- **Power Query:** The ETL tool where we apply final touches.



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

